Лабораторная работа
Численное интегрирование методом Симпсона с использованием OpenMP

Студент группы ИА332 Рубцов Роман Кириллович

Новосибирск, 2026

1 Цель работы

Разработка параллельной программы численного интегрирования произвольной подынтегральной функции методом Симпсона с использованием технологии OpenMP. Обеспечение заданной апостериорной точности вычислений с помощью правила Рунге. Проведение экспериментального анализа масштабируемости программы при различном числе потоков выполнения.

2 Конфигурация

- Процессор (CPU): Intel Core i5-10400F (базовая частота 2.90 ГГц);
- Конфигурация ядер: 6 физических ядер / 12 логических потоков;
- Кэш-память:
  - Кэш первого уровня (L1): суммарно 384 КБ (по 64 КБ на ядро: 32 КБ для данных + 32 КБ для инструкций);
  - Кэш второго уровня (L2): суммарно 1.5 МБ (по 256 КБ на ядро);
  - Кэш третьего уровня (L3): 12 МБ общего кэша Intel Smart Cache.
- Оперативная память (RAM): DDR4 16 гб
- Операционная система: Arch Linux (x86_64);

3 Теоретические сведения

3.1 Метод Симпсона

Метод Симпсона является одним из наиболее точных методов численного интегрирования для гладких функций. Формула Симпсона для приближенного вычисления определенного интеграла $\int_a^b f(x)\,dx$ при равномерном разбиении интервала $[a;b]$ на n равных частей (n — четное) имеет вид:

$$S_n = \frac{h}{3}\left[f(a) + 4\sum_{i=1,3,\ldots}^{n-1} f(x_i) + 2\sum_{i=2,4,\ldots}^{n-2} f(x_i) + f(b)\right], \tag{1}$$

где $h = \frac{b-a}{n}$ — шаг интегрирования, $x_i = a + i \cdot h$.

Погрешность метода Симпсона оценивается как $O(h^4)$, что обеспечивает высокую скорость сходимости при увеличении числа узлов разбиения.

3.2 Правило Рунге для оценки погрешности

Для автоматического выбора шага интегрирования используется правило Рунге. Если I_n и I_{2n} — приближенные значения интеграла, полученные при разбиении на n и $2n$ частей соответственно, то оценка погрешности вычисляется по формуле:

$$\varepsilon \approx \frac{|I_{2n} - I_n|}{2^p - 1}, \tag{2}$$

где $p = 4$ — порядок точности метода Симпсона. Таким образом, условие достижения заданной точности ε имеет вид:

$$\frac{|I_{2n} - I_n|}{15} < \varepsilon. \tag{3}$$

В данной работе $\varepsilon = 10^{-6}$.

3.3 Параллелизация с использованием OpenMP

Основная вычислительная нагрузка в методе Симпсона приходится на суммирование значений функции в узлах сетки. Данный цикл может быть эффективно распараллелен с помощью директивы `#pragma omp parallel for reduction(+ : sum)`, которая распределяет итерации цикла между потоками и автоматически выполняет редукцию (суммирование) частных результатов.

4 Постановка задачи

Необходимо написать программу численного интегрирования функции:

$$f(x) = \frac{x^4}{0.5x^2 + x + 6} \tag{4}$$

на интервале $[a; b] = [0.4; 1.5]$ методом Симпсона с апостериорной оценкой погрешности по правилу Рунге ($\varepsilon = 10^{-6}$). Программа должна быть реализована с использованием OpenMP.

Требуется провести экспериментальный анализ масштабируемости программы при числе потоков: 1, 2, 4, 8, 16, 32, 64, 128. Время выполнения измеряется как среднее геометрическое из 100 прогонов для повышения статистической достоверности результатов.

5 Результаты выполнения

5.1 Программная реализация

Программа написана на языке C++ с использованием библиотеки OpenMP. Алгоритм работы следующий:

1. Задается начальное число разбиений $n = 2000000$.
2. Вычисляется значение интеграла I_n функцией `simpson()`.
3. В цикле удваивается число разбиений ($n \leftarrow 2n$), вычисляется новое значение I_{2n} и проверяется условие сходимости по правилу Рунге: $|I_{2n} - I_n|/15 < 10^{-6}$.
4. Цикл завершается при достижении требуемой точности.

Функция `simpson()` содержит параллельный цикл с директивой `reduction(+ : sum)`, обеспечивающий корректное суммирование вкладов от всех потоков. Для каждого количества потоков выполняется 100 независимых прогонов, результаты усредняются геометрически (через логарифмы) для устойчивости к выбросам.

5.2 Экспериментальные результаты

Результаты измерений времени выполнения представлены в таблице 1 и на рисунке 1.

Таблица 1: Результаты измерения времени выполнения программы

Число потоков	**Среднее геом. время (нс)**
1	7 904 990
2	4 016 640
4	2 120 370
8	1 968 590
16	1 821 950
32	1 606 260
64	1 676 690
128	1 810 780

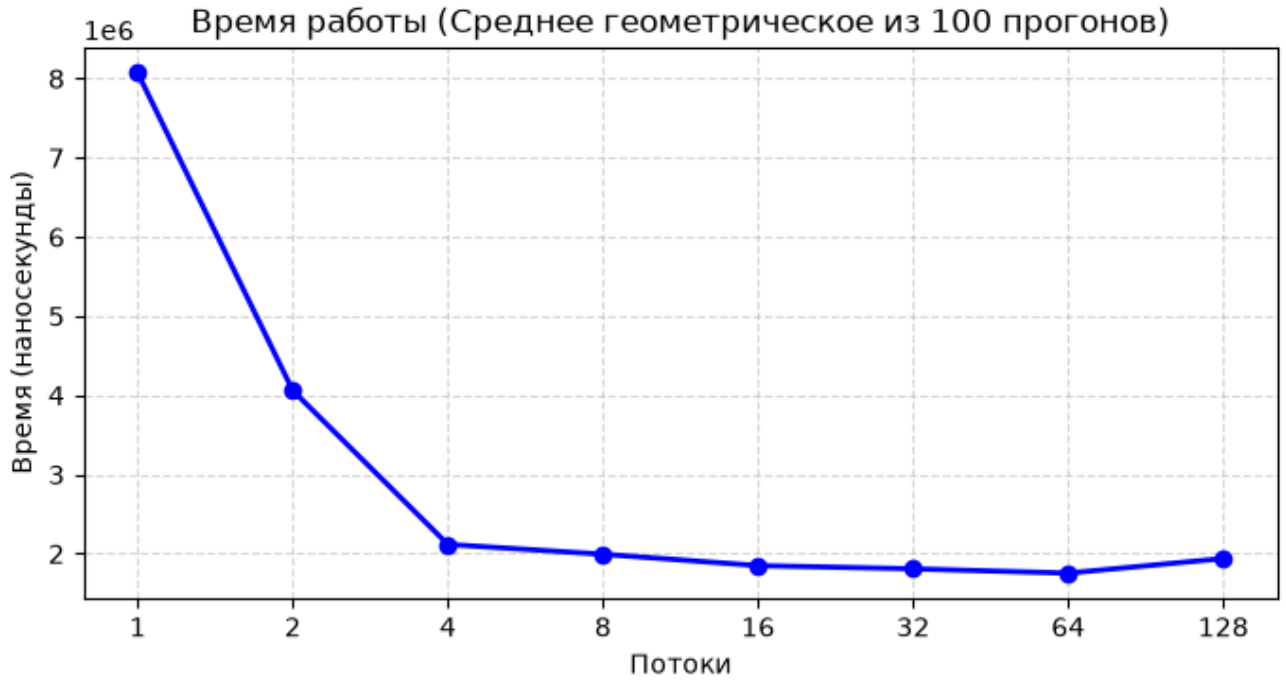


Рис. 1: Зависимость времени выполнения от числа потоков (среднее геометрическое из 100 прогонов)

5.3 Анализ масштабируемости

На рисунке 1 и в таблице 1 наблюдаются следующие закономерности:

- **Область эффективного ускорения (1–4 потока):** При переходе от 1 к 2 потокам время сокращается почти вдвое (с ≈ 7.9 мкс до ≈ 4.0 мкс), что соответствует близкому к линейному ускорению. При переходе к 4 потокам время уменьшается еще примерно вдвое (до ≈ 2.1 мкс). Это свидетельствует об эффективной параллелизации основной вычислительной нагрузки — цикла суммирования в методе Симпсона.

- **Область насыщения (4–32 потока):** Начиная с 4 потоков, дальнейшее увеличение их числа дает всё меньший эффект. Время снижается незначительно: с 2.12 мкс (4 потока) до 1.61 мкс (32 потока). Ускорение замедляется из-за накладных расходов на создание и синхронизацию потоков, а также из-за конкуренции за ресурсы процессора (кэш-память, шина памяти).

- **Область деградации (64–128 потоков):** При 64 и особенно при 128 потоках наблюдается рост времени выполнения по сравнению с 32 потоками (1.68 мкс и 1.81 мкс соответственно). Это объясняется тем, что количество потоков превышает число физических ядер процессора, что приводит к избыточным переключениям контекста, contention за общие ресурсы и дополнительным накладным расходам планировщика операционной системы.

Максимальное ускорение достигается при 32 потоках и составляет:

$$S_{max} = \frac{T_1}{T_{32}} = \frac{7\,904\,990}{1\,606\,260} \approx 4.92. \tag{5}$$

6 Выводы

В ходе выполнения практической работы были получены следующие результаты:

1. **Реализация алгоритма:** Разработана программа численного интегрирования функции $f(x) = x^4/(0.5x^2+x+6)$ на интервале $[0.4; 1.5]$ методом Симпсона с автоматическим выбором шага по правилу Рунге ($\varepsilon = 10^{-6}$). Параллелизация выполнена с использованием OpenMP (директива `parallel for reduction`).

2. **Точность вычислений:** Использование правила Рунге с коэффициентом 1/15 (для метода 4-го порядка) позволило автоматически достичь заданной апостериорной точности 10^{-6} без ручного подбора числа разбиений. Начальное значение $n = 2000000$ обеспечило быструю сходимость.

3. **Масштабируемость:** Экспериментальный анализ показал:

   - Близкое к линейному ускорение в диапазоне 1–4 потоков;
   - Замедление роста производительности при 8–32 потоках из-за накладных расходов параллелизации;
   - Деградацию производительности при 64–128 потоках вследствие превышения числа физических ядер и роста contention за ресурсы.

4. **Оптимальное число потоков:** Для данной задачи оптимальным является использование 32 потоков, при которых достигается максимальное ускорение $S \approx 4.92$. Дальнейшее увеличение числа потоков нецелесообразно.

5. **Методология измерений:** Использование среднего геометрического из 100 прогонов вместо арифметического среднего позволило получить более робастные оценки времени выполнения, устойчивые к случайным выбросам, вызванным фоновыми процессами операционной системы.